Towards Type Systems That Reinterpret Themselves

Pavel Sokolov[1] ‹ppsokolov@hse.ru›
Alexander Kuklev[2] ‹a@kuklev.com›

[1]HSE University, Laboratory of Theoretical Computer Science
[2]Radboud Univ. Nijmegen, Department of Software Science

October 20, 2026

Abstract

We search for type systems capable of "eating itself" in the sense that their own metatheoretical properties could be stated and proven inside it. In this particular instance, we study consequences of introducing a reinterpretation operator $m[\![t]\!]$ which allows instantiation of a closed term t as a construction inside a model m definable internally. Our main result is a design of a two-level type theory with reinterpretation, $[\![2]\!]\mathsf{LTT}$, where reintrepretation can be used to provide a limited form of universe polymorphism, prove external parametricity and do some synthetic mathematics.

1 Introduction

An idea that a "type theory should eat itself" is a well-established concept in study of dependent type theory. Under this guise, if a type theory is presented as a "full-scale system for constructive mathematics" [CITE], then it should be capable of proving its own metatheoretical properties. For example, in a paper bearing the same name [CITE], the author presents a normalizer for type theory written in Agda [CITE].

Of course, Gödel's incompleteness theorems [CITE] still apply in a constructive context, so naïvely proving self-consistency of a type system supporting primitive recursion and induction is out of question. What is usually proven instead is that a particular implementation of a type theory inside itself is sound assuming the consistency of the theory, so no paradox arises [CITE] [CITE]; this is still useful because it leads to the phenomena of *certified proof assistants*.

Another possible approach is to work in a two-level type theory [CITE]. As the name implies, such a theory consists of two levels: the lower *object* level is the main type theory, and a higher *meta* level is used to internalize statements about object level which are external to it. A classical example given in the cited paper is a construction of a *semisimplicial type of level* n for each external $n \in \mathbb{N}$; object level here is homotopy type theory (HoTT) with univalence axiom (UA) [CITE] and meta level is Martin-Löf type theory (MLTT) with uniqueness of identity proofs (UIP) [CITE]; in usual type theory, these two axioms are mutually exclusive, but the trick is that an object-level equality becomes a different type when viewed from meta level, so UA and UIP apply to different notions of "equality", keeping the system consistent (and sound with existing models of HoTT [CITE]).

We aim to generalize this construction in the following way. It can be said that an object level has a fixed *interpretation* in a meta-level; what if the system allowed to vary the interpretation? In other words, what if the system included a generic interpretation operator denoted $m[\![t]\!]$, where t is a term which is being interpreted, and m is a model definable inside the system? (We will call such systems *reflective*.)

Turns out, we would get quite a few useful metatheoretical results about object-level theory for free; in particular, Wadler's parametricity theorem [CITE] (for its formulation in dependently typed context, see [CITE] and [CITE]) about object-level theory would be provable on the meta-level by means of specifying a relational model; a fraction of displayed type theory [CITE] can be obtained via similar translation of a type into a unary predicate. Another fun consequence is that object-level theory

would get universe polymorphism via displacements as described by Conor Titania McBride [CITE] and Favonia [CITE]; we suspect that there might be other useful applications of internal interpretation.

1.1 Contributions

- So our main goal is to construct an implementable reflective type system. The main difficulty in constructing such a system arises from a certain no-go result discussed in Section 2, which forces us to limit the power of reinterpretation in order to keep type checking decidable.
- For ease of presentation, we first offer a reflective type system based on the Hindley-Milner type system, where object level is simply-typed lambda calculus with product types (STLC), called $[\![\mathsf{S}]\!]\mathsf{TLC}$ (Section 3). We prove that it is conservative with respect to STLC (hence strongly normalizing and sound), and that global type inference is decidable (Section 3.1).
- Building upon the insights gathered from developing $[\![\mathsf{S}]\!]\mathsf{TLC}$, we are finally equipped to dive into the reflective version of two-level type theory, where object level is vanilla MLTT, dubbed $[\![2]\!]\mathsf{LTT}$ (Section 4). We prove soundness and conservativity of it as well (Section 4.2).
- As intended, by virtue of reinterpretation, $[\![2]\!]\mathsf{LTT}$ incorporates both a restricted form of universe polymorphism and external parametricity. And, of course, you can use it as a system for synthetic mathematics with internal reuse of synthetic results (Section 4.3).

2 No-go theorem of internal interpretation

So we want to build a reflective type system. That is, a system where a user is able to both define a model of a system internally and apply it to well-typed terms to get their internal interpretations. Naturally, we would like user-defined models to respect definitional equalities of a system; in a dependently typed system, this could be naturally achieved by using an identity type to spell out corresponding equalities inside a model. Then, it would be nice if, before interpreting a term, system would first bring it to normal form to get rid of the artifacts of the construction in the result of the interpretation. Finally, in a dependently typed system, it is natural to expect reductions of open terms, so we would like our interpretation to compute on open terms.

The theorem proven below asserts that we cannot have all three in a single system.

First, a couple of definitions. We're going to work within a classical type-theoretical framework, which introduces a sort of contexts Γ, terms t, t' and types T linked by the typing relation $\Gamma \vdash t : T$; computation is specified by a one-step reduction relation $t \rightsquigarrow t'$ on terms.

Definition 1 (Naïve computational interpretation)**.** We call a type operator $F[_]$ a naïve computational interpretation of types if it respects computation on types (that is, if there is a reduction relation on types ($\rightsquigarrow$) and $T \rightsquigarrow T'$, then $F[T] \rightsquigarrow^* F[T']$), and computes as a primitive recursive function on normal forms of types. Analogously, term with a hole $[\![_]\!]$ is a naïve computational interpretation of terms if it respects computation on terms, computes as a primitive recursive function on normal forms of terms and, if $\Gamma \vdash t : T$, then $\Gamma \vdash [\![t]\!] : F[T]$.

Theorem 1 (No-go theorem of naive interpretation)**.** *In a confluent type system with a type of functions and naïve computational interpretation, if reduction commutes with substitution, for arbitrary well-typed s and t there exists u such that:*

$$\mathtt{app}[\![\lambda x.t]\!][\![s]\!] \rightsquigarrow^* u \ {}^*\!\!\leftsquigarrow [\![t[s/x]]\!]$$

where app *is how $[\![_]\!]$ acts on applications.*

Therefore, $\mathtt{app}[\![\lambda x.t]\!][\![s]\!]$ and $[\![t[s/x]]\!]$ are definitionally equivalent, which forces beta-reductions in a model to hold definitionally rather than propositionally, directly contradicting our desired outcome. Analogous theorems can be proven for any other reduction present in the system; in other words, **propositional equality is not strong enough**. The proof is by direct example:

Proof. We are given well-typed $\Gamma \vdash s : S$ and $\Gamma, x : S \vdash t : T$. Consider the following term:

$$\Gamma, x : S \to T, y : S \vdash [\![x\, y]\!] : F[T]$$

Term under interpretation is in normal form, hence the whole term computes to $\mathtt{app}[\![x]\!][\![y]\!]$. Substituting $\lambda x.t$ for x and s for y, we get $\mathtt{app}[\![\lambda x.t]\!][\![s]\!]$.

On the other hand, if we perform substitution first, we would get $\Gamma \vdash [\![(\lambda x.t)s]\!] : F[T]$. Since naïve interpretation respects reduction, this reduces to $[\![t[s/x]]\!]$.

Since reduction commutes with substitution and system is confluent, there should be such u that both terms reduce to. □

Fundamentally, the proof above exploits the openness of the original term; if we restrict interpretation to compute on closed terms only, we can arrive at a sound system without requiring definitional equalities in models, as exemplified by our non-dependently typed system $[\![\mathsf{S}]\!]\mathsf{TLC}$ described in Section 3. And this, actually, is a well-known setup: under the same conditions (no dependent types + computation on closed terms), self-interpreters for System T and System Fω were developed [7] [3].

For a dependently typed system, however, our no-go theorem leaves us with three options:

1. Restrict models to those where equalities hold definitionally rather than propositionally.
2. Do not proceed with interpretation unless the term is closed. To allow reasoning on open terms, equations describing the behavior of interpretation are exposed as propositional axioms.
3. Do not reduce open terms under interpretation at all, performing the interpretation as-is. To simplify the result of interpretation, one can rewrite it with equalities present in the model.

If you view existing work on displacement [18, 19], internal parametricity [1] and displayed type theory [16] through the lens of interpretation inside a fixed model (or family of models), you'll see that corresponding models indeed validate equalities definitionally; thus it seems that the first approach needs to be taken. It is also the most satisfying from the computational point of view. However, if you want a sound system where models can be user-defined, you need to constrain models through typing discipline. By following techniques currently known to us [5], constraining models in such a way would require defining a type of models as a type of algebras of an inductive-inductive-recursive type (or even inductive-inductive-recursive simplicial type, for univalent type theories) which is well beyond the scope of this paper.

Out of the two remaining options, the last one is better suited for practical dependently-typed systems since it properly computes on open terms. In addition, it doesn't restrict the class of possible models while at the same time providing the benefits of the first approach: if a model is such that the equations hold definitionally, no rewriting on the result of interpretation is needed to reduce it. The only downside is that, possibly, reductions *after* interpretation are more costly than the ones we could perform *before* it.

3 Almost simply typed almost reflective language: $[\![\mathsf{S}]\!]\mathsf{TLC}$

We start with a simpler language to iron out the details and actually figure out how to properly reinterpret terms internally before moving on to a dependently typed system. The simplest language for which we know a lot of interesting models is the simply typed lambda calculus with products, which we refer to as "STLC" throughout this paper; in this section, we'll present a language capable of interpreting STLC terms as constructions in STLC models that can be defined in our language.

So, what are the models of STLC? In addition to the usual set-theoretic semantics where $S \to T$ is interpreted as a function from S to T and $S \times T$ is a cartesian product, there is proof-theoretic semantics that relate terms of type T to intuitionistic proofs of proposition $[\![T]\!]$, where $(\to)$ is now understood to mean implication, and $(\times)$ is conjunction. This, of course, is a classic Curry-Howard correspondence [14]; both set-theoretic and proof-theoretic semantics are generalized by a complete description of STLC models as given by Joachim Lambek [17]. According to this *Curry-Howard-Lambek* correspondence, models of STLC are precisely the cartesian closed categories (or CCCs for short). Definitions vary; we're going to use the definition 2.

Definition 2 (Cartesian closed category)**.** A cartesian closed category is the following datum:

1. A class[1] of *objects* Ob and a family of *morphisms* $\mathrm{Hom}(S,T)$ indexed by $S,T \in \mathrm{Ob}$, where $f \in \mathrm{Hom}(S,T)$ is denoted by $f : S \to T$. In addition, S is called the *source* of f, and T is the *target* of f.

2. For each $A \in \mathrm{Ob}$ there's an *identity morphism* $\mathrm{id}_A : A \to A$; for each $f : A \to B$ and $g : B \to C$, there's their *composition* $g \circ f : A \to C$. If a source-target of an identity morphism is clear from the context, the subscript is elided. Also, the following properties hold:

$$\mathrm{id} \circ f = f = f \circ \mathrm{id} \qquad\qquad (f \circ g) \circ h = f \circ (g \circ h)$$

   whenever all compositions are well-defined.

3. There's a *terminal object* $\top \in \mathrm{Ob}$ such that for each $A \in \mathrm{Ob}$ there's a **unique** morphism $!_A : A \to \top$.

4. For each $A, B \in \mathrm{Ob}$ there's their *product* $A \times B$ together with *projections* $\pi_1 : A \times B \to A$ and $\pi_2 : A \times B \to B$; for each object $C \in \mathrm{Ob}$ and a pair of morphisms $f : C \to A$ and $g : C \to B$, there's a **unique** *pairing morphism* $[f; g] : C \to A \times B$ such that the following properties hold:

$$\pi_1 \circ [f; g] = f \qquad\qquad \pi_2 \circ [f; g] = g$$

5. For each $A, B \in \mathrm{Ob}$ there's their *exponential object* B^A together with *evaluation morphism* $\mathrm{ev} : B^A \times A \to B$; for each object $C \in \mathrm{Ob}$ and morphism $f : C \times A \to B$, there's a **unique** *abstraction* $\lambda f : C \to B^A$ such that the following property holds for arbitrary $g : C \to A$:

$$\mathrm{ev} \circ [\lambda f; g] = f \circ [\mathrm{id}_C; g]$$

Then, a well-typed term $\Gamma \vdash t : T$ is interpreted as a morphism $[\![t]\!] : [\![\Gamma]\!] \to [\![T]\!]$, where $[\![\Gamma]\!]$ is a left-nested product of interpretations of types from Γ:

$$[\![x : A, y : B, z : C]\!] = ((\top \times [\![A]\!]) \times [\![B]\!]) \times [\![C]\!].$$

Unit type is now a terminal object; product of types is now a product of objects; arrow type is an exponential object. Equations in (2) are needed to validate the structural properties of substitution; equations in (4) correspond to reductions of pairs; equation in (5) validates β-reduction. **Unique**ness properties correspond to η-equivalence.

For example, in our language, $m\,[\![t]\!]$ is the operator interpreting the term t in an internal model m. On a sample term

$$\cdot \vdash \lambda f x y.(f x, y) : (A \to B) \to A \to C \to B \times C$$

it computes like this:

$$m\,[\![\lambda f x y.(f x, y))]\!] = \\ = \mathtt{lam}\,(\mathtt{lam}\,(\mathtt{lam}\,(\mathtt{pair}\,(\mathtt{eval} \circ \mathtt{pair}\,(\mathtt{proj}_2 \circ \mathtt{proj}_1 \circ \mathtt{proj}_1)\,(\mathtt{proj}_2 \circ \mathtt{proj}_1))\,\mathtt{proj}_2)))$$

Where `lam`, `pair` and `eval` are the abstraction morphism, the pairing morphism, and the evaluation morphism taken from m, defined in the same way as in the definition 2, translated into types of our system.

Now, moving on to the actual design of our language. As discussed in Section 2, for the non-dependent case we are now dealing with, we will interpret only normal forms of closed terms. Among other things, this means that we actually don't need equations and **unique**nesses from Definition 2 to hold in our internal models! This approach doesn't break any of the metatheoretical properties

[1] Objects do not necessarily form a set: for example, one might want to have a category of sets where morphisms are functions between them. To deal with sizing issues, pick appropriate set-theoretical foundations you are most comfortable with (NBG, MK, ZFC/S or something else); this detail is not important in our development

MonoType	S, T, C, P, E, TT	$::=$	α	type variable
		$\mid$	$\top$	unit type
		$\mid$	$S \times T$	product type
		$\mid$	$S \to T$	arrow type
MonoCtx	Δ	$::=$	$\cdot$	empty context
		$\mid$	$\Delta, x \colon T$	context extension
PolyType	τ	$::=$	T	monomorphic type
		$\mid$	$\forall \alpha . \tau$	forall type
PolyCtx	Γ	$::=$	$\cdot$	empty context
		$\mid$	$\Gamma, x \colon \tau$	context extension
Ctx	Ξ	$::=$	$\Gamma \parallel \Delta$	joined context
Term	s, t	$::=$	x	variable
		$\mid$	$()$	singleton
		$\mid$	(s, t)	pair
		$\mid$	$\pi_1 \, t$	first projection
		$\mid$	$\pi_2 \, t$	second projection
		$\mid$	$\lambda x . t$	abstraction
		$\mid$	$t \, s$	application
		$\mid$	$\mathtt{let} \, x = s \, \mathtt{in} \, t$	polymorphic binding
		$\mid$	$m[\![x]\!]$	reinterpretation
Model	m	$::=$	$\langle T, t, \ldots \rangle$	model def. (see typing rules)

Figure 1: [[S]]TLC terms and values

considered in Section 3.1. Additionally, for the sake of example, we're only going to consider models where objects are types. Then, the internalization of definition 2 would require 1) type operators to define types of morphisms, products, and exponential objects; 2) polymorphic functions to operationalize the $\forall\exists$ introductions of constructions in a model. Actually, we're going to get rid of type operators as well, since we can do with just types-in-a-context which are then substituted into the definition of a model for checking term-level operations. This way, our language is a small extension of the classic Hindley-Milner type system; grammar rules are collected in a Figure 1. You'll notice that, despite our initial description, grammar doesn't define a reinterpretation operator $m[\![t]\!]$ for arbitrary term t; instead, only variables (denoted by a metavariable x) get to be reinterpreted. In this system, reinterpretation is nothing but a total function on closed values which is called whenever a value is substituted into such an x (so, in a way, this is similar to hereditary substitution [CITE]).

Definition 3 (Values of [[S]]TLC)**.** Values are denoted by metavariables u, v in the BNF grammar below. Metavariable n denotes neutral terms:

$$u, v ::= n \mid () \mid (u, v) \mid \lambda x . v$$
$$n ::= x \mid \pi_i \, n \mid n \, v$$

Thus, to define interpretation, it is enough to consider STLC terms:

Definition 4 (Interpretation of terms in [[S]]TLC)**.** Given a model m, a list of variables γ and a term t, interpretation of a term $m[\![\gamma \vdash t]\!]$ is a recursive function defined by the following equations:

$$m[\![\gamma \vdash ()]\!] = \mathtt{cnst} \tag{1}$$
$$m[\![\gamma \vdash (s, t)]\!] = \mathtt{pair} \, m[\![\gamma \vdash s]\!] \, m[\![\gamma \vdash t]\!] \tag{2}$$
$$m[\![\gamma \vdash \pi_i \, t]\!] = \mathtt{comp} \, \mathtt{proj}_i \, m[\![\gamma \vdash t]\!] \tag{3}$$
$$m[\![\gamma \vdash x]\!] = \gamma[x] \tag{4}$$
$$m[\![\gamma \vdash \lambda x . t]\!] = \mathtt{lam} \, m[\![(\gamma, x) \vdash t]\!] \tag{5}$$
$$m[\![\gamma \vdash t \, s]\!] = \mathtt{comp} \, \mathtt{eval} \, (\mathtt{pair} \, m[\![\gamma \vdash t]\!] \, m[\![\gamma \vdash s]\!]) \tag{6}$$

$$\frac{s \rightsquigarrow s'}{(s,t) \rightsquigarrow (s',t)} \qquad \frac{t \rightsquigarrow t'}{(s,t) \rightsquigarrow (s,t')} \qquad \frac{t \rightsquigarrow t'}{\pi_i t \rightsquigarrow \pi_i t'} \qquad \pi_i(t_1,t_2) \rightsquigarrow t_i$$

$$\frac{t \rightsquigarrow t'}{\lambda x.t \rightsquigarrow \lambda x.t'} \qquad \frac{s \rightsquigarrow s'}{s\,t \rightsquigarrow s'\,t} \qquad \frac{t \rightsquigarrow t'}{s\,t \rightsquigarrow s\,t'} \qquad (\lambda x.t)s \rightsquigarrow t[s/x]$$

$$\frac{s \rightsquigarrow s'}{\mathtt{let}\,x = s\,\mathtt{in}\,t \rightsquigarrow \mathtt{let}\,x = s'\,\mathtt{in}\,t} \qquad \frac{t \rightsquigarrow t'}{\mathtt{let}\,x = s\,\mathtt{in}\,t \rightsquigarrow \mathtt{let}\,x = s\,\mathtt{in}\,t'} \qquad \frac{\mathrm{fv}(v) = \varnothing}{\mathtt{let}\,x = v\,\mathtt{in}\,t \rightsquigarrow t[v /\!\!/ x]}$$

Figure 2: ⟦S⟧TLC computation rules

where model operations `cnst`, `pair` and others are taken from m and $\gamma[x]$ is also defined by equations:

$$(\gamma, x)[x] = \mathtt{proj}_2 \tag{7}$$
$$(\gamma, y)[x] = \mathtt{comp}\,\gamma[x]\,\mathtt{proj}_1 \tag{8}$$

Here is our way to enforce a principle that *only closed terms get interpreted*: once a term s bound by a polymorphic let-binding $\mathtt{let}\,x = s\,\mathtt{in}\,t$ becomes closed and is brought into normal form, only then is it substituted into t, and whenever it is substituted into $m[\![x]\!]$, it immediately gets reinterpreted. This operation is, as noted before, similar to hereditary substitution; it is denoted by $t[s /\!\!/ x]$ in our computation rules given in Figure 2. So it acts like a regular substitution in all cases except for variables under reinterpretation:

$$m[\![x]\!][s /\!\!/ x] = m[\![() \vdash s]\!]$$

One can easily notice that computation would get stuck if let-bound terms were to contain variables coming from unapplied λ-abstractions. Our typing rules given in Figure 3 protect us from that, allowing to refer only to let-bound terms inside other let-bound terms. Again, aside from that and the typing rule for universes, there's nothing different from the usual declarative typing rules for a Hindley-Milner type system [9].

To check models, we abuse our type system a bit to avoid adding type operators and complicating it: types of morphisms C, product objects P and exponentials E are specified as open monomorphic types which can use two type variables, 0 and 1. This fact is reflected in the syntax of a model using notation $\Lambda 01.C$, $\Lambda 01.P$ and $\Lambda 01.E$ respectively. 0 and 1 are then filled in by substitution when we need to type model operations (notation $T[S_1, S_2]$ is a shorthand for $T[S_1/0, S_2/1]$). The terminal object is just a type named TT.

To check reinterpretation, we need to define how the model acts on monomorphic types. Since there's no computation on them, reinterpretation of types is once again defined as a recursive function on syntax:

Definition 5 (Interpretation of types in ⟦S⟧TLC)**.** Let m be a correct model and σ be a substitution of monomorphic types. Then, the interpretation of type in a model is the following function:

$$m[\![\sigma \vdash \alpha]\!] = \sigma\alpha \tag{9}$$
$$m[\![\sigma \vdash \top]\!] = TT \tag{10}$$
$$m[\![\sigma \vdash (S \times T)]\!] = P[m[\![\sigma \vdash S]\!], m[\![\sigma \vdash T]\!]] \tag{11}$$
$$m[\![\sigma \vdash S \to T]\!] = E[m[\![\sigma \vdash S]\!], m[\![\sigma \vdash T]\!]]. \tag{12}$$

3.1 Metatheoretic properties of ⟦S⟧TLC

Here we only study some basic properties of a system to make sure that everything works as expected: computations are confluent; types are preserved by all computations; all of the types can be inferred if you wish to do so; everything terminates, and, modulo reduction of `let`-bindings, this is the same old

$$\Xi \vdash () : \top \qquad \frac{\Xi \vdash s : S \qquad \Xi \vdash t : T}{\Xi \vdash (s,t) : S \times T} \qquad \frac{\Xi \vdash t : S \times T}{\Xi \vdash \pi_1\, t : S} \qquad \frac{\Xi \vdash t : S \times T}{\Xi \vdash \pi_2\, t : T}$$

$$\frac{x : T \in \Delta}{\Gamma \parallel \Delta \vdash x : T} \qquad \frac{\Gamma \parallel \Delta, x : S \vdash t : T}{\Gamma \parallel \Delta \vdash \lambda x.t : S \to T} \qquad \frac{\Xi \vdash t : S \to T \qquad \Xi \vdash s : S}{\Xi \vdash t\, s : T}$$

$$\frac{\Gamma \vdash s : S \qquad \mathrm{fv}(S) = \vec{\alpha} \qquad \Gamma, x : \forall \vec{\alpha}.S \parallel \Delta \vdash t : T}{\Gamma \parallel \Delta \vdash \mathtt{let}\, x = s\, \mathtt{in}\, t : T}$$

$$\frac{\Gamma \parallel \Delta \vdash m\,\mathrm{model} \qquad x : \forall \vec{\alpha}.T \in \Gamma \qquad \sigma = \left[\vec{S}/\vec{\alpha}\right]}{\Gamma \parallel \Delta \vdash m[\![x]\!] : C_m[TT_m, m[\![\sigma \vdash T]\!]]}$$

$$\frac{\begin{array}{c}\mathrm{fv}(C) \cup \mathrm{fv}(P) \cup \mathrm{fv}(E) \subseteq \{0,1\} \qquad \Xi \vdash \mathtt{id} : C[\alpha,\alpha] \qquad \Xi \vdash \mathtt{comp} : C[\beta,\gamma] \to C[\alpha,\beta] \to C[\alpha,\gamma] \\ \mathrm{fv}(TT) = \varnothing \qquad \Xi \vdash \mathtt{cnst} : C[\alpha, TT] \qquad \Xi \vdash \mathtt{pair} : C[\gamma,\alpha] \to C[\gamma,\beta] \to C[\gamma, P[\alpha,\beta]] \\ \Xi \vdash \mathtt{proj}_1 : C[P[\alpha,\beta],\alpha] \qquad \Xi \vdash \mathtt{proj}_2 : C[P[\alpha,\beta],\beta] \\ \Xi \vdash \mathtt{lam} : C[P[\gamma,\beta],\alpha] \to C[\gamma, E[\alpha,\beta]] \qquad \Xi \vdash \mathtt{eval} : C[P[E[\alpha,\beta],\beta],\alpha]\end{array}}{\Xi \vdash \langle \Lambda 01.C, \mathtt{id}, \mathtt{comp}, TT, \mathtt{cnst}, \Lambda 01.P, \mathtt{pair}, \mathtt{proj}_1, \mathtt{proj}_2, \Lambda 01.E, \mathtt{lam}, \mathtt{eval}\rangle\,\mathrm{model}}$$

Figure 3: ⟦S⟧TLC declarative typing rules

STLC we started with. Proofs mostly follow standard techniques since the rules for reinterpretation are structured to directly plug into proofs of properties of the STLC and the Hindley-Milner type system.

Theorem 2 (Church-Rosser theorem for ⟦S⟧TLC)**.** *Reflexive transitive closure of* $(\rightsquigarrow)$ *has the Church-Rosser property.*

Proof. Classic proof via parallel reduction relation [CITE]. Parallel reduction is defined as:

$$t \rightrightarrows t \qquad \frac{s \rightrightarrows s' \qquad t \rightrightarrows t'}{(s,t) \rightrightarrows (s',t')} \qquad \frac{t \rightrightarrows t'}{\pi_i\, t \rightrightarrows \pi_i\, t'} \qquad \frac{t_1 \rightrightarrows t_1'}{\pi_1(t_1,t_2) \rightrightarrows t_1'} \qquad \frac{t_2 \rightrightarrows t_2'}{\pi_2(t_1,t_2) \rightrightarrows t_2'}$$

$$\frac{t \rightrightarrows t'}{\lambda x.t \rightrightarrows \lambda x.t'} \qquad \frac{s \rightrightarrows s' \qquad t \rightrightarrows t'}{s\, t \rightrightarrows s'\, t'} \qquad \frac{s \rightrightarrows s' \qquad t \rightrightarrows t'}{(\lambda x.s)\, t \rightrightarrows s'[t'/x]}$$

$$\frac{s \rightrightarrows s' \qquad t \rightrightarrows t'}{\mathrm{let}\, x = s\, \mathrm{in}\, t \rightrightarrows \mathrm{let}\, x = s'\, \mathrm{in}\, t'} \qquad \frac{t \rightrightarrows t'}{\mathrm{let}\, x = v\, \mathrm{in}\, t \rightrightarrows t'[v \mathbin{/\!\!/} x]}$$

□

Theorem 3 (Type preservation for ⟦S⟧TLC)**.** *The following is true:*

1. *If* $\Xi \vdash m\,\mathrm{model}$, $\Delta \vdash t : T$ *and* δ *is a list of variables from* Δ*, then* $\Xi \vdash m[\![\delta \vdash t]\!] : C_m[m[\![\Delta]\!], m[\![T]\!]]$.
2. *If* $\Gamma \parallel \Delta \vdash s : S$ *and* $\Gamma \parallel \Delta, x : S, \Delta' \vdash t : T$*, then* $\Gamma \parallel \Delta, \Delta' \vdash t[s/x] : T$.
3. *If* $\Gamma \vdash v : \tau$ *and* $\Gamma, x : \tau, \Gamma' \parallel \Delta \vdash t : T$*, then* $\Gamma, \Gamma' \parallel \Delta \vdash t[v \mathbin{/\!\!/} x] : T$.
4. *If* $\Gamma \vdash t : T$ *and* $t \rightsquigarrow t'$*, then* $\Gamma \vdash t' : T$.

Proof. By structural induction on terms and $(\rightsquigarrow)$. □

Theorem 4 (Strong normalization for ⟦S⟧TLC)**.** *Closed terms typable in* ⟦S⟧TLC *are strongly normalizing with respect to* $(\rightsquigarrow)$ *and reduce to a value.*

Proof. Classic proof via saturated sets [CITE]. Let SN be a set of terms which are strongly normalizing with respect to $(\rightsquigarrow)$ and normal forms of which are values. Let Atomic be the least set of terms satisfying the following conditions: variables x are Atomic; if t is Atomic, then so are $\pi_i\, t$ and $t\, s$. Then, let $t \to_{\mathrm{WHR}} t'$ be the following relation on terms:

$$\frac{t \in \mathsf{SN}}{(\lambda x.s)t \to_{\mathrm{WHR}} s[t/x]} \qquad \frac{t_2 \in \mathsf{SN}}{\pi_1(t_1, t_2) \to_{\mathrm{WHR}} t_1} \qquad \frac{t_1 \in \mathsf{SN}}{\pi_2(t_1, t_2) \to_{\mathrm{WHR}} t_2} \qquad \frac{t \to_{\mathrm{WHR}} t'}{t\, s \to_{\mathrm{WHR}} t'\, s}$$

$$\frac{t \to_{\mathrm{WHR}} t'}{\pi_i\, t \to_{\mathrm{WHR}} \pi_i\, t'} \qquad \frac{}{\mathtt{let}\, x = v\, \mathtt{in}\, t \to_{\mathrm{WHR}} t[v \mathbin{/\!\!/} x]}(\mathrm{fv}(v) = \varnothing)$$

Finally, let SAT be the following family of sets of terms:

- if $X \in \mathsf{SAT}$, then $X \subseteq \mathsf{SN}$;
- if $X \in \mathsf{SAT}$, then $\mathsf{Atomic} \cap \mathsf{SN} \subseteq \mathsf{SAT}$;
- if $X \in \mathsf{SAT}$, $t' \in X$ and $t \to_{\mathrm{WHR}} t'$, then $t \in X$.

Let there be the following relation $\mathcal{R}^{\zeta}_{\tau} \in \mathsf{SAT}$, where ζ is a mapping from type variables into SAT:

- $t \in \mathcal{R}^{\zeta}_{\alpha}$ iff $t \in \zeta(\alpha)$;
- $t \in \mathcal{R}^{\zeta}_{\top}$ iff $t \in \mathsf{SN}$;
- $t \in \mathcal{R}^{\zeta}_{T_1 \times T_2}$ iff $t \in \mathsf{SN}$ and $\pi_i\, t \in \mathcal{R}^{\zeta}_{T_i}$ for $i = 1, 2$;
- $t \in \mathcal{R}^{\zeta}_{S \to T}$ iff $t \in \mathsf{SN}$ and for all $s \in \mathcal{R}^{\zeta}_{S}$, $t\, s \in \mathcal{R}^{\zeta}_{T}$;
- $t \in \mathcal{R}^{\zeta}_{\forall \alpha.T}$ iff $t \in \mathsf{SN}$ and $t \in \mathcal{R}^{\zeta, \alpha \mapsto X}_{T}$ for all $X \in \mathsf{SAT}$.

Indeed, $\mathcal{R}^{\zeta}_{\tau} \in \mathsf{SAT}$. Furthermore, the following property holds: if $t \in \mathcal{R}^{\zeta}_{\forall \vec{\alpha}.T}$, then $m[\![() \vdash \mathrm{nf}(t)]\!] \in \mathcal{R}^{\zeta}_{m[\![\vec{S}/\vec{\alpha}] \vdash T]\!]}$ for all m s.t. all terms inside it are in corresponding $\mathcal{R}^{\zeta}_{\forall}$.

Then we can prove our main lemma: if $\Gamma \vdash t : T$ and γ is a substitution s.t. $\gamma\, x \in \mathcal{R}^{\zeta}_{\Gamma(x)}$ for all $x \in \Gamma$, then $\gamma\, t \in \mathcal{R}^{\zeta}_{T}$. □

Theorem 5 (Conservativity over STLC)**.** *Normal forms of closed terms typable in* $[\![\mathsf{S}]\!]\mathsf{TLC}$ *are typable in STLC.*

Proof. Thanks to type preservation, normal form $\mathrm{nf}(t)$ is typable in $[\![\mathsf{S}]\!]\mathsf{TLC}$. By definition, it is a value; therefore, it has no reinterpretation subterms and let-bindings, hence is typable in STLC. □

Theorem 6. *Global type inference for* $[\![\mathsf{S}]\!]\mathsf{TLC}$ *is decidable.*

Proof. It is enough to extend any unification-based type inference algorithm for the Hindley-Milner type system to support models and reinterpretation.

- Since type operators in models are provided explicitly, it is enough to *check* that their fields are typable with expected types.
- Since polymorphic functions do not depend on the local context, their type is fully resolved when they are added to the context.
- For case $m[\![x]\!]$, and $(x : \forall \vec{\alpha}.T) \in \Gamma$, we can directly compute $m[\![[\vec{\eta}/\vec{\alpha}] \vdash T]\!]$ (since m is known and T is fully resolved) where $\vec{\eta}$ are fresh metavariables.

□

Thus, in this section, we have introduced a sound type system based on the Hindley-Milner type system which is capable of interpreting its own closed terms in an arbitrary "cartesian closed category" definable inside it. This system is not truly *reflective* since models are not fully internal; they require a special form of type operators to work, which is not accessible to the rest of the type system. To internalize model definition, we move on to a dependently typed system in the next section.

4 Almost reflective dependently typed language: $[\![2]\!]\mathsf{LTT}$

Now we are ready to take on the main challenge: design a minimal dependently typed reflective language, and provide internal models of itself to yield universe polymorphism, external parametricity and do synthetic mathematics inside it. As a starting point, we take Martin-Löf type theory with $\Pi-$,$\Sigma-$, Id$-$ types, natural number type and infinite non-cumulative hierarchy of universes. As discussed in Section 2, the best practical solution to reinterpretation here would be to forbid reductions under it and reinterpret terms as-is, and constrain models using Id-types. Still, in comparison to non-dependently typed case, there are infinitely more problems to solve if we want to define a truly reflective system:

1. If we naïvely adopt an approach to typing of models from Section 3, in order to specify a model, we would need to specify countably many models, one per each universe. And even if we did, to which universe does the type of models belong?

2. Now that we have computations in types, we have to deal with conversion rule under reinterpretation:
$$\frac{t : T \quad T \equiv T'}{t : T'} \overset{?}{\rightsquigarrow} \frac{[\![t]\!] : [\![T]\!] \quad [\![T]\!] \equiv [\![T']\!]}{[\![t]\!] : [\![T']\!]}$$
The problem is that, after interpretation, $[\![T]\!]$ is only *propositionally* equal to $[\![T']\!]$.

3. If we want a truly reflective system, we need to know what to do with *reinterpretation of reinterpretation*:
$$\lambda x.[\![[\![x]\!]_1]\!]_2 \quad \text{and} \quad [\![\lambda x.[\![x]\!]_1]\!]_2$$
In the first case, we perform two subsequent interpretations on a yet unknown x; in the second case, we interpret the process of interpretation itself. What should this mean?

To deal with universe sizing issues, consider the following observation: in a system without level binders, each term uses only a finite number of levels. Hence to reinterpret this term, we need to only provide a finite number of universe models, which can together fit into some larger universe of a finite level. No universe polymorphism required! However, we now have to track the maximal level which appears in a term in our typing judgements.

To interpret conversion rule, we start by first pinpointing the locations where conversion rule is actually used; in order to do this ergonomically, we use bidirectional type checking. At the moment of conversion, interpretation has enough information to know both T and T'; thus we can translate a chain of reductions performed during definitional equality check into a chain of propositional equalities taken from coherence conditions proven in the definition of a model. Then, a point of conversion is wrapped in equality elimination with synthesized witness as a scrutinee.

In reinterpreting reinterpretation, we actually only deal with "double reinterpretation": cases like $\lambda x.[\![[\![x]\!]_1]\!]_2$ could be dealt with by *merging* two models into a new one, a technique reminiscent of *Collapsing towers of interpreters* [2]; it is not required in our system since the innermost reinterpretation is allowed to compute in this case. As for interpreting interpretation itself as in $[\![\lambda x.[\![x]\!]_1]\!]_2$, we explicitly forbid such constructions by introducing a family of modalities $\Box_\kappa$ meaning "no interpretation was used in construction of this term, and all levels appearing in it are not above κ". Terms have to have a boxed type to be reinterpreted. In the notation for modality and general typing discipline for it, we largely follow Gratzer et al [12] with a few important differences discussed later. We also use $\Box$ to relax a ban on computations inside reinterpretation by allowing "splicing" boxed expressions into other boxed expressions; inside splices, computation is allowed to bring terms to the boxed form for further interpretation. Such a typing discipline brings us very close to two-level type theories [CITE], but we found notation via modalities to be more convenient; such presentation also puts an accent on closeness of our system to metaprogramming systems with staging such as MetaOCaml and the like [15].

Grammar rules given in Figure 4 reflect discussion above. Note that there is no separate type of models $\mathcal{M}$: instead of there being a single builtin type, there is a family of model types $\mathcal{M}^{\ell'_0,\dots,\ell'_\kappa}_{\ell_0,\dots,\ell_\kappa}$ where each concrete type is *definable* in $[\![2]\!]\mathsf{LTT}$; types in a family are indexed by a bunch of universe levels whose exact role will be explained later; a general idea is that ℓ_j is a level which types in $\mathcal{U}_j$ would have after interpretation in such a model.

Context	Γ	$::=$	$\cdot$	empty context
		$\mid$	$\Gamma, x : T$	variable extension
		$\mid$	$\Gamma.\,\ell🔒$	context locking
Type	T, S	$::=$	$\mathcal{U}_\ell$	universe
		$\mid$	$\mathrm{El}(t)$	universal type family
		$\mid$	Nat_ℓ	natural number type
		$\mid$	$(x : S) \times T$	Σ-type
		$\mid$	$(x : S) \to T$	Π-type
		$\mid$	$t_1 =_T t_2$	identity type
		$\mid$	$\Box_\ell\, T$	type of interpretable terms
		$\mid$	$t\,[\![T]\!]$	interpretation of types
Level	ℓ, κ, j	$\in$	$\mathbb{N}$	universe levels
Term	t, s	$::=$	$x \mid y \mid z \mid \ldots$	variables
		$\mid$	$\mathtt{U}_\ell$	code of a universe
		$\mid$	$\mathtt{Nat}_\ell$	code of a natural number type
		$\mid$	0	zero
		$\mid$	$\mathrm{S}\,t$	successor
		$\mid$	$\mathrm{natind}_{x.T}\,t\,\{s_0, s_S\}$	induction
		$\mid$	$\mathtt{Sigma}(x : s)t$	code of a Σ-type
		$\mid$	(s, t)	pairing
		$\mid$	$\pi_1\,t \mid \pi_2\,t$	projections
		$\mid$	$\mathtt{Pi}(x : s)t$	code of a Π-type
		$\mid$	$\lambda x.\,t$	abstraction
		$\mid$	$t\,s$	application
		$\mid$	$\mathtt{Id}\,t\,t_1\,t_2$	code of an identity type
		$\mid$	refl	reflexivity
		$\mid$	$\mathrm{J}_{xy.S}\,t\,s$	equality elimination
		$\mid$	$\mathtt{Box}_\ell\,t$	code of a box type
		$\mid$	$[t]_🔒$	locking
		$\mid$	$[t]_{🔓}$	unlocking
		$\mid$	$t\,[\![s]\!]$	interpretation of terms
		$\mid$	$t : T$	ascription

Figure 4: $[\![2]\!]\mathsf{LTT}$ grammar

$$\textsc{WF-Nil}\quad \cdot \vdash_\ell \qquad \textsc{WF-Var}\quad \frac{\Gamma \vdash_\ell \qquad \Gamma \vdash_\ell T : \kappa}{\Gamma, x : T \vdash_\ell}(\kappa \leqslant \ell) \qquad \textsc{WF-Lock}\quad \frac{\Gamma \vdash_\ell \qquad \text{🔒} \notin \Gamma}{\Gamma.\,\ell\text{🔒} \vdash_\kappa} \qquad \textsc{Level-El}\quad \frac{\Gamma \vdash_\ell t \Rightarrow \mathcal{U}_\kappa}{\Gamma \vdash_\ell \mathrm{El}(t) : \kappa}(\kappa + 1 \leqslant \ell)$$

$$\textsc{Level-Uni}\quad \frac{\Gamma \vdash_\ell}{\Gamma \vdash_\ell \mathcal{U}_\kappa : \kappa + 1} \qquad \textsc{Level-Nat}\quad \frac{\Gamma \vdash_\ell}{\Gamma \vdash_\ell \mathrm{Nat}_\kappa : \kappa} \qquad \textsc{Level-Sigma}\quad \frac{\Gamma \vdash_\ell S : j \qquad \Gamma, x : S \vdash_\ell T : \kappa}{\Gamma \vdash_\ell (x : S) \times T : j \sqcup \kappa}$$

$$\textsc{Level-Pi}\quad \frac{\Gamma \vdash_\ell S : j \qquad \Gamma, x : S \vdash_\ell T : \kappa}{\Gamma \vdash_\ell (x : S) \to T : j \sqcup \kappa} \qquad \textsc{Level-Id}\quad \frac{\Gamma \vdash_\ell T : \kappa \qquad \Gamma \vdash_\ell t_1 \Leftarrow T \qquad \Gamma \vdash_\ell t_2 \Leftarrow T}{\Gamma \vdash_\ell t_1 =_T t_2 : \kappa}$$

$$\textsc{Level-Box}\quad \frac{\text{🔒} \notin \Gamma \qquad \Gamma.\,\ell\text{🔒} \vdash_\kappa T : j}{\Gamma \vdash_\ell \Box_\kappa T : \kappa}(j \leqslant \kappa) \qquad \textsc{Level-TyInterp}\quad \frac{\Gamma \vdash_\ell t \Rightarrow \mathcal{M}^{\ell'_0,\ldots,\ell'_\kappa}_{\ell_0,\ldots,\ell_\kappa} \qquad \text{🔒} \notin \Gamma \qquad \Gamma.\ell\text{🔒} \vdash_\kappa T : j}{\Gamma \vdash_\ell t[\![T]\!] : \ell_j}(j \leqslant \kappa)$$

Figure 5: Context well-formedness & level placement rules

Typing rules are also a straightforward adaptation of a general recipe for bidirectional typing of dependently-typed systems [11] and are given in Figures 5-6. Important departure from standard bidirectional recipe "constructors are checked, eliminators are inferred" is that levels of types are always inferred since we do not have cumulativity of universes and do not have a lifting operator, thus we cannot check levels of Σ and Π. Also note that exact model type is always inferred, both in interpretation of types and in interpretation of terms: even though $\mathcal{M}$ would be some huge nested Σ-type, it is still more correct to *match* on it instead of asking to provide levels explicitly.

As already said, reinterpretation is constrained to act on boxed types; in addition, we require a non-locked context for reinterpretation. Together these constraints guarantee that, indeed, boxed terms do not contain reinterpretation subterms. An important departure from MLTT🔒 [12], aside from adding a family of modalities, lies in the way context is locked in $[_]_{\text{🔒}}$ and unlocked in $[_]_{\text{🔓}}$: while locking, we require the outer context to be unlocked; while unlocking, instead of erasing all locks, we erase the context up to the first lock. Effectively, this enforces a constraint that typing context never contains more than a single lock. Thus, for example, our $\Box_\ell$ is not a comonad, but axiom K from modal logic still holds[2]:

$$\frac{\cdot \vdash_\ell A : \ell_A \quad \cdot \vdash_\ell B : \ell_B}{\cdot \vdash_\kappa \lambda f x.[[f]_{\text{🔓}}[x]_{\text{🔓}}]_{\text{🔒}} : \Box_\ell(A \to B) \to \Box_\ell A \to \Box_\ell B}(\ell_A \sqcup \ell_B \leqslant \ell)$$

In this way, unlocking truly gains the semantics of "splicing" in the sense that it can only contain boxed values from the "outer world".

4.1 Computation rules and models of dependent type theories

Note that discussion above was ambient to the choice of formalism to present models. Now it is finally time to fix the formalism since we're going to need it to define the type of models $\mathcal{M}$.

In line with current state of the art, we use Categories-with-Families [8] (or CwFs for short) as it is currently one of the most convenient frameworks to specify models of dependent type theories. Here is a self-contatined definition of a category with families, given for clear reference in subsequent internalization:

Definition 6 (Category)**.** A category is the following datum[3]:

[2]Interestingly enough, a similar K-modality is first described by Birkedal and others in "Modal dependent type theory and dependent right adjoints" [6]. Coincidentally, they refine typing judgements with universe levels, too! They have a single modality, though.

[3]Of course, cartesian closed category (2) is also a category. We repeat parts of definitions for self-containment reasons.

$$\textsc{Infer-Uni}\ \frac{\Gamma \vdash_\ell}{\Gamma \vdash_\ell \mathtt{U}_\kappa \Rightarrow \mathcal{U}_{\kappa+1}} \qquad \textsc{Infer-Nat}\ \frac{\Gamma \vdash_\ell}{\Gamma \vdash_\ell \mathtt{Nat}_\kappa \Rightarrow \mathcal{U}_\kappa} \qquad \textsc{Check-Zero}\ \frac{\Gamma \vdash_\ell}{\Gamma \vdash_\ell 0 \Leftarrow \mathrm{Nat}_\kappa} \qquad \textsc{Check-Succ}\ \frac{\Gamma \vdash_\ell t \Leftarrow \mathrm{Nat}_\kappa}{\Gamma \vdash_\ell \mathrm{S}\, t \Leftarrow \mathrm{Nat}_\kappa}$$

$$\textsc{Infer-NatInd}\ \frac{\Gamma \vdash_\ell t \Rightarrow \mathrm{Nat}_\kappa \qquad \Gamma, x : \mathrm{Nat}_\kappa \vdash_\ell T : j \qquad \Gamma \vdash_\ell s_0 \Leftarrow T[(0 : \mathrm{Nat}_\kappa)/x] \qquad \Gamma \vdash_\ell s_S \Leftarrow (y : \mathrm{Nat}_\kappa) \to (z : T[y/x]) \to T[(\mathrm{S}\, y : \mathrm{Nat}_\kappa)/x]}{\Gamma \vdash_\ell \mathrm{natind}_{x.T}\, t\, \{s_0, s_S\} \Rightarrow T[t/x]} (j \leqslant \ell)$$

$$\textsc{Infer-Sigma}\ \frac{\Gamma \vdash_\ell s \Rightarrow \mathcal{U}_j \qquad \Gamma, x : \mathrm{El}(s) \vdash_\ell t \Rightarrow \mathcal{U}_\kappa}{\Gamma \vdash_\ell \mathtt{Sigma}(x : s)t \Rightarrow \mathcal{U}_{j \sqcup \kappa}} \qquad \textsc{Check-Pair}\ \frac{\Gamma \vdash_\ell s \Leftarrow S \qquad \Gamma \vdash_\ell t \Leftarrow T[(s : S)/x]}{\Gamma \vdash_\ell (s, t) \Leftarrow (x : S) \times T}$$

$$\textsc{Infer-Proj}_1\ \frac{\Gamma \vdash_\ell t \Rightarrow (x : S) \times T}{\Gamma \vdash_\ell \pi_1\, t \Rightarrow S} \qquad \textsc{Infer-Proj}_2\ \frac{\Gamma \vdash_\ell t \Rightarrow (x : S) \times T}{\Gamma \vdash_\ell \pi_2\, t \Rightarrow T[(\pi_1\, t)/x]} \qquad \textsc{Infer-Pi}\ \frac{\Gamma \vdash_\ell s \Rightarrow \mathcal{U}_j \qquad \Gamma, x : \mathrm{El}(s) \vdash_\ell t \Rightarrow \mathcal{U}_\kappa}{\Gamma \vdash_\ell \mathtt{Pi}(x : s)t \Rightarrow \mathcal{U}_{j \sqcup \kappa}}$$

$$\textsc{Check-Abs}\ \frac{\Gamma, x : S \vdash_\ell t \Leftarrow T}{\Gamma \vdash_\ell \lambda x.\, t \Leftarrow (x : S) \to T} \qquad \textsc{Infer-App}\ \frac{\Gamma \vdash_\ell t \Rightarrow (x : S) \to T \qquad \Gamma \vdash_\ell s \Leftarrow S}{\Gamma \vdash_\ell t\, s \Rightarrow T[(s : S)/x]}$$

$$\textsc{Infer-Id}\ \frac{\Gamma \vdash_\ell t \Rightarrow \mathcal{U}_\kappa \qquad \Gamma \vdash_\ell t_1 \Leftarrow \mathrm{El}(t) \qquad \Gamma \vdash_\ell t_2 \Leftarrow \mathrm{El}(t)}{\Gamma \vdash_\ell \mathtt{Id}\, t\, t_1\, t_2 \Rightarrow \mathcal{U}_\kappa} \qquad \textsc{Check-Refl}\ \frac{t_1 \equiv t_2}{\Gamma \vdash_\ell \mathrm{refl} \Leftarrow t_1 =_T t_2}$$

$$\textsc{Infer-J}\ \frac{\Gamma \vdash_\ell t \Rightarrow t_1 =_T t_2 \qquad \Gamma, x : T, y : t_1 =_T x \vdash_\ell S : \kappa \qquad \Gamma \vdash_\ell s \Leftarrow S[(t_1 : T)/x, (\mathrm{refl} : t_1 =_T t_1)/y]}{\Gamma \vdash_\ell \mathrm{J}_{xy.S}\, t\, s \Rightarrow S[(t_2 : T)/x, t/y]} (\kappa \leqslant \ell) \qquad \textsc{Infer-Var}\ \frac{\🔒 \notin \Delta \qquad \Gamma, x : T, \Delta \vdash_\ell}{\Gamma, x : T, \Delta \vdash_\ell x \Rightarrow T}$$

$$\textsc{Infer-Box}\ \frac{\🔒 \notin \Gamma \qquad \Gamma.\,_\ell\🔒 \vdash_\kappa t \Rightarrow \mathcal{U}_j}{\Gamma \vdash_\ell \mathtt{Box}_\kappa\, t \Rightarrow \mathcal{U}_\kappa} (j \leqslant \kappa) \qquad \textsc{Check-Lock}\ \frac{\🔒 \notin \Gamma \qquad \Gamma.\,_\ell\🔒 \vdash_\kappa t \Leftarrow T}{\Gamma \vdash_\ell [t]_🔒 \Leftarrow \Box_\kappa T} \qquad \textsc{Infer-Unlock}\ \frac{\🔒 \notin \Delta \qquad \Gamma \vdash_\kappa t \Rightarrow \Box_j T}{\Gamma.\,_\kappa🔒, \Delta \vdash_\ell [t]_🔓 \Rightarrow T} (j \leqslant \ell)$$

$$\textsc{Infer-Interp}\ \frac{\🔒 \notin \Gamma \qquad \Gamma \vdash_\ell s \Rightarrow \mathcal{M}^{\ell'_0,\ldots,\ell'_k}_{\ell_0,\ldots,\ell_k} \qquad \Gamma \vdash_\ell t \Rightarrow \Box_\kappa T}{\Gamma \vdash_\ell s[\![t]\!] \Rightarrow s[\![T]\!]} \qquad \textsc{Infer-Ascr}\ \frac{\Gamma \vdash_\ell T : \kappa \qquad \Gamma \vdash_\ell t \Leftarrow T}{\Gamma \vdash_\ell t : T \Rightarrow T} (\kappa \leqslant \ell)$$

$$\textsc{Check-Inferred}\ \frac{\Gamma \vdash_\ell t \Rightarrow S \qquad S \equiv T}{\Gamma \vdash_\ell t \Leftarrow T}$$

Figure 6: Bidirectional term typing rules

1. A class of *objects* Ob and a family of *morphisms* $\mathrm{Hom}(S,T)$ indexed by $S,T \in \mathrm{Ob}$, where $f \in \mathrm{Hom}(S,T)$ is denoted by $f : S \to T$. In addition, S is called *source* of f, while T is *target* of f.

2. For each $A \in \mathrm{Ob}$ there's an *identity morphism* $\mathrm{id}_A : A \to A$; for each $f : A \to B$ and $g : B \to C$, there's their *composition* $g \circ f : A \to C$. If a source-target of an identity morphism is clear from the context, the subscript is elided. Also, the following properties hold:

$$\mathrm{id} \circ f = f = f \circ \mathrm{id} \qquad\qquad (f \circ g) \circ h = f \circ (g \circ h)$$

   whenever all compositions are well-defined.

Definition 7 (Terminal object)**.** Terminal object of category $\mathcal{C}$ is an object $\top \in \mathrm{Ob}_{\mathcal{C}}$ such that for each $A \in \mathrm{Ob}_{\mathcal{C}}$ there's a **unique** morphism $!_A : A \to \top$.

Definition 8 (Presheaf)**.** A presheaf P on a category $\mathcal{C}$ (written as $P : \mathcal{C}^{\mathrm{op}} \to \mathrm{Set}$) is a mapping which takes objects of $\mathcal{C}$ to sets, and morphisms of $\mathcal{C}$ to functions between sets, so that if $f : A \to_{\mathcal{C}} B$, then $P(f)$ is a function from $P(B)$ to $P(A)$. Also, the following properties hold:

$$P(\mathrm{id}_A) = \mathrm{id}_{P(A)} \qquad\qquad P(g \circ f) = P(f) \circ P(g)$$

Where $\mathrm{id}_{P(A)}$ is an identity function of a set $P(A)$ and $P(f) \circ P(g)$ is a composition of functions $P(f)$ and $P(g)$.

Definition 9 (Category of elements)**.** Given a presheaf $P : \mathcal{C}^{\mathrm{op}} \to \mathrm{Set}$, a category of elements $\int_{\mathcal{C}} P$ is a category where objects are pairs (A, x) such that $A \in \mathrm{Ob}_{\mathcal{C}}$ and $x \in P(A)$, and morphisms from (A, x) to (B, y) are morphisms $f : A \to_{\mathcal{C}} B$ such that $P(f)(y) = x$.

Definition 10 (Category with families)**.** A category with families is the following datum:

1. A *category of contexts* $\mathcal{C}$ which is a category with a terminal object;

2. A *presheaf of types* $\mathrm{Ty} : \mathcal{C}^{\mathrm{op}} \to \mathrm{Set}$. Given a morphism $\gamma : \Gamma \to_{\mathcal{C}} \Delta$ and a *type* $T \in \mathrm{Ty}(\Delta)$, element $\mathrm{Ty}(\gamma)(T) \in \mathrm{Ty}(\Gamma)$ is usually written as $T[\gamma]$ for brevity.

3. A *presheaf of terms* $\mathrm{Tm} : \left(\int_{\mathcal{C}} \mathrm{Ty}\right)^{\mathrm{op}} \to \mathrm{Set}$. Same as with types, $\mathrm{Tm}(\gamma)(t) \in \mathrm{Tm}(\Gamma, T)$ is usually written as $t[\gamma]$ for brevity.

4. For each $\Gamma \in \mathrm{Ob}_{\mathcal{C}}$ and $T \in \mathrm{Ty}(\Gamma)$, there is an *extended context* $\Gamma.T \in \mathrm{Ob}_{\mathcal{C}}$ together with a *weakening morphism* $p : \Gamma.T \to \Gamma$ and *variable element* $q : \mathrm{Tm}(\Gamma.T, T[p])$. In addition, for all Δ, $\gamma : \Delta \to \Gamma$ and $t \in \mathrm{Tm}(\Delta, T[\gamma])$, there is a unique *extended substitution* $\langle \gamma, t \rangle : \Delta \to \Gamma.T$ such that the following holds:

$$p \circ \langle \gamma, t \rangle = \gamma \qquad\qquad q[\langle \gamma, t \rangle] = t$$

A definition of a Cwf (def. 10) clearly captures the bare-bones structure of a dependently typed theory: objects of $\mathcal{C}$ represent contexts, while morphisms represent substitutions mapping one context to another. Substitution also acts both on types and terms; given a correct type T in context Γ, one can extend a context to gain new context $\Gamma, x : T$ in which there's a well-typed term $x : T$; there's a way to obtain substitutions into an extended context by providing a term suitable for x.

This base definition can be readily extended to incorporate type (and term) formers of any system. A CwF for $[\![2]\!]\mathsf{LTT}$, then, has to include Ty- and Tm- formers for analogues of universes, Π- and Σ-types, natural numbers and identity types. Interpretation for $\Box_\ell$ is not needed since we succesfully prevented boxed types from appearing inside boxed terms via rigid 🔒-ing discipline[4]. Thus, contexts which need to be interpreted are constructed via vanilla context extensions from empty context, and a usual CwF is enough for our purposes. Since actual internalized definition of our CwF largely repeats

[4]That's also the reason we dubbed this system *almost* reflective, since its internal reinterpretation doesn't cover modalities and reinterpretation itself.

$$\frac{t \rightsquigarrow t'}{\mathrm{El}(t) \rightsquigarrow \mathrm{El}(t')} \qquad \mathrm{El}(\mathtt{U}_\kappa) \rightsquigarrow \mathcal{U}_\kappa \qquad \mathrm{El}(\mathtt{Nat}_\kappa) \rightsquigarrow \mathrm{Nat}_\kappa \qquad \mathrm{El}(\mathtt{Sigma}(x : s)t) \rightsquigarrow (x : \mathrm{El}(s)) \times \mathrm{El}(t)$$

$$\mathrm{El}(\mathtt{Pi}(x : s)t) \rightsquigarrow (x : \mathrm{El}(s)) \to \mathrm{El}(t) \qquad \mathrm{El}(\mathtt{Id}\, t\, t_1\, t_2) \rightsquigarrow t_1 =_{\mathrm{El}(t)} t_2 \qquad \mathrm{El}(\mathtt{Box}_\kappa t) \rightsquigarrow \Box_\kappa \mathrm{El}(t)$$

$$\frac{S \rightsquigarrow S'}{(x : S) \times T \rightsquigarrow (x : S') \times T} \qquad \frac{T \rightsquigarrow T'}{(x : S) \times T \rightsquigarrow (x : S) \times T'} \qquad \frac{S \rightsquigarrow S'}{(x : S) \to T \rightsquigarrow (x : S') \to T}$$

$$\frac{T \rightsquigarrow T'}{(x : S) \to T \rightsquigarrow (x : S) \to T'} \qquad \frac{T \rightsquigarrow T'}{t_1 =_T t_2 \rightsquigarrow t_1 =_{T'} t_2} \qquad \frac{t_1 \rightsquigarrow t_1'}{t_1 =_T t_2 \rightsquigarrow t_1' =_T t_2}$$

$$\frac{t_2 \rightsquigarrow t_2'}{t_1 =_T t_2 \rightsquigarrow t_1 =_T t_2'} \qquad t[\![T]\!] \rightsquigarrow t.\mathrm{Tm}(t.\cdot, t[\![\cdot \vdash T]\!])$$

Figure 7: $[\![2]\!]\mathsf{LTT}$ typal computation rules

typing and computation rules from Figures 5-6 and Figures 7-8 in internal language, we do not provide it here; to get a bit more details, you can refer to our Appendix A.

Finally, on to computation rules as given in Figures 7-8. Mostly they are standard MLTT computation rules for bidirectional typing (without η-equality for now). Importantly enough, we **do not compute inside locked terms and inside boxed types** in order to not stumble into a variant of our no-go theorem 1 again (modal variant is spelled out in discussion, see Section 5.2.2).

Also we need to give the rules for computing reinterpretation; as it turns out, *reinterpreting terms as-is* allows to define reinterpretation not as a redex, but rather as a couple of total functions on types and terms, just as it was in a non-dependently-typed case (Section 3). To be precise, we need two mutually recursive functions for terms: $m[\![\Gamma \vdash t \Leftarrow T]\!]$ computes on checkable terms, while $m[\![\Gamma \vdash t \Rightarrow]\!]$ computes on synthesizing terms and outputs an inferred type; both compute elements of corresponding $\mathrm{Tm}(\Gamma, _)$-s from a CwF m. $m[\![\Gamma \vdash T]\!]$ computes an element of $\mathrm{Ty}(\Gamma)$. Another auxiliary function is required to compute chain of equalities to handle conversion rule; this function has two subtle points:

- At this point, we don't know if system is strongly normalizing yet, therefore we potentially might encounter an infinite chain of equalities.
- In systems where equality is proof-relevant (and we want to be able to develop in that direction), different reduction chains might lead to different equality witnesses, so such a function is potentially not well-defined.

Our solution is the following: let us fix a certain evaluation strategy (for example, strict evaluation) and employ it in normalization. Since terms we reduce to find a witness are vanilla MLTT terms, they are strongly normalizing, so witness extraction terminates.

TODO: provide interpretation definition.

4.2 Metatheoretical properties of $[\![2]\!]\mathsf{LTT}$

Once again, we only try to make sure of the most essential properties: that computations are confluent, that types during computations are preserved, all computations terminate and that resulting type system is a conservative extension of MLTT.

Theorem 7 (Church-Rosser theorem for $[\![2]\!]\mathsf{LTT}$)**.** *Reflexive transitive closure of* $(\rightsquigarrow)$ *has Church-Rosser property.*

Proof. Classic proof via parallel reduction relation [CITE]. Since interpretation computes as a recursive functions on open terms, it commutes with substitution, which is crucial for confluence.

TODO: write out parallel reduction? □

$$\frac{t \rightsquigarrow t'}{S\,t \rightsquigarrow S\,t'} \qquad \frac{t \rightsquigarrow t'}{\mathrm{natind}_{x.T} t\{s_0, s_S\} \rightsquigarrow \mathrm{natind}_{x.T} t'\{s_0, s_S\}}$$

$$\mathrm{natind}_{x.T}(0 : \mathrm{Nat}_\ell)\{s_0, s_S\} \rightsquigarrow (s_0 : T[(0 : \mathrm{Nat}_\ell)/x])$$

$$\mathrm{natind}_{x.T}(S\,t : \mathrm{Nat}_\ell)\{s_0, s_S\} \rightsquigarrow (s_S : (x : \mathrm{Nat}_\ell) \to T \to T[(Sx)/x])\,t\,(\mathrm{natind}_{x.T}(t : \mathrm{Nat}_\ell)\{s_0, s_S\})$$

$$\frac{s \rightsquigarrow s'}{\mathtt{Sigma}(x : s)t \rightsquigarrow \mathtt{Sigma}(x : s')t} \qquad \frac{t \rightsquigarrow t'}{\mathtt{Sigma}(x : s)t \rightsquigarrow \mathtt{Sigma}(x : s)t'} \qquad \frac{s \rightsquigarrow s'}{(s, t) \rightsquigarrow (s', t)} \qquad \frac{t \rightsquigarrow t'}{(s, t) \rightsquigarrow (s, t')}$$

$$\frac{t \rightsquigarrow t'}{\pi_i t \rightsquigarrow \pi_i t'} \qquad \pi_i((t_1, t_2) : (x : S) \times T) \rightsquigarrow t_i \qquad \frac{s \rightsquigarrow s'}{\mathtt{Pi}(x : s)t \rightsquigarrow \mathtt{Pi}(x : s')t} \qquad \frac{t \rightsquigarrow t'}{\mathtt{Pi}(x : s)t \rightsquigarrow \mathtt{Pi}(x : s)t'}$$

$$\frac{t \rightsquigarrow t'}{\lambda x.t \rightsquigarrow \lambda x.t'} \qquad \frac{t \rightsquigarrow t'}{t\,s \rightsquigarrow t'\,s} \qquad \frac{s \rightsquigarrow s'}{t\,s \rightsquigarrow t\,s'} \qquad ((\lambda x.t) : (x : S) \to T)s \rightsquigarrow t[(s : S)/x]$$

$$\frac{t \rightsquigarrow t'}{\mathtt{Id}\,t\,t_1\,t_2 \rightsquigarrow \mathtt{Id}\,t'\,t_1\,t_2} \qquad \frac{t_1 \rightsquigarrow t_1'}{\mathtt{Id}\,t\,t_1\,t_2 \rightsquigarrow \mathtt{Id}\,t\,t_1'\,t_2} \qquad \frac{t_2 \rightsquigarrow t_2'}{\mathtt{Id}\,t\,t_1\,t_2 \rightsquigarrow \mathtt{Id}\,t\,t_1\,t_2'} \qquad \frac{t \rightsquigarrow t'}{\mathrm{J}_{xy.S}\,t\,s \rightsquigarrow \mathrm{J}_{xy.S}\,t'\,s}$$

$$\mathrm{J}_{xy.S}(\mathrm{refl} : t_1 =_T t_2)\,s \rightsquigarrow (s : S[t_2/x, (\mathrm{refl} : t_1 =_T t_2)/y]) \qquad \frac{t \rightsquigarrow t'}{(t : T) \rightsquigarrow (t' : T)} \qquad \frac{t \rightsquigarrow t'}{[t]_{🔓} \rightsquigarrow [t']_{🔓}}$$

$$[[t]_{🔒} : \Box_\kappa T]_{🔓} \rightsquigarrow t : T \qquad [[t]_{🔓}]_{🔒} \rightsquigarrow t \qquad \frac{s \rightsquigarrow s'}{s[\![t]\!] \rightsquigarrow s'[\![t]\!]} \qquad \frac{t \rightsquigarrow t'}{s[\![t]\!] \rightsquigarrow s[\![t']\!]}$$

$$s[\![[t]_{🔒} : \Box_\kappa T]\!] \rightsquigarrow s[\![\cdot \vdash t \Leftarrow T]\!] \qquad \frac{}{t : T \rightsquigarrow t}(t \text{ is synthesizing})$$

Figure 8: $[\![2]\!]\mathsf{LTT}$ term computation rules

Theorem 8 (Type preservation for $[\![2]\!]\mathsf{LTT}$)**.** *If* $\Gamma \vdash t \Leftarrow T$ *and* $t \rightsquigarrow t'$*, then* $\Gamma \vdash t' \Leftarrow T$*; analogously for* $\Gamma \vdash t \Rightarrow T$*.*

Proof. Note that, analogous to other dependently typed systems, a context reduction lemma holds which is crucial to handle reduction in domain of Π-types (where both $\Leftarrow$ and $\Rightarrow$ can stand in place of a colon):

$$\frac{\Gamma, x : S, \Delta \vdash_\ell t : T \quad S \rightsquigarrow S'}{\Gamma, x : S', \Delta \vdash_\ell t : T}$$

Then the proof proceeds by structural induction on terms and $(\rightsquigarrow)$. Note that, in a bidirectional system, one has to take care to structure rules in such a way that substitutions substitute terms that are inferred, so that inferrable terms stay inferrable and checkable terms stay checkable.

An important difference from other dependently typed systems is, of course, interpretation. For this case, analogous to the proof of type preservation for $[\![\mathsf{S}]\!]\mathsf{TLC}$ (Section 3.1), a vital lemma holds:

$$\frac{\Gamma.\ell\text{🔒}, \Delta \vdash_\kappa t \Leftarrow T \quad \Gamma \vdash_\ell s \Rightarrow \mathcal{M}^{\ell'_0,\ldots,\ell'_\kappa}_{\ell_0,\ldots,\ell_\kappa}}{\Gamma \vdash_\ell s[\![\Delta \vdash t \Leftarrow T]\!] \Rightarrow s.\mathrm{Tm}(s[\![\Delta]\!], s[\![\Delta \vdash T]\!])}$$

Along with similar lemmas for $_[\![_ \vdash _ \Rightarrow]\!]$ and $_[\![_ \vdash _]\!]$. □

Theorem 9 (Identity model)**.** *For each* ℓ*, there is an identity model* $I_\ell : \mathcal{M}^{1,\ldots,\ell+1}_{0,\ldots,\ell}$ *with a property that* $I_\ell[\![[t]_\text{🔒} : \Box_\kappa T]\!] \equiv t : T$ *for each closed* t*.*

Proof. Take a universe at appropriate level as type of contexts; $\mathrm{Ty}_\ell(\Gamma) = \Gamma \to \mathcal{U}_\ell$; $\mathrm{Tm}_\ell(\Gamma, T) = (\gamma : \Gamma) \to T\,\gamma$. Define all of the remaining term interpretations to reinterpret to the same terms. □

Theorem 10 (Strong normalization for $[\![2]\!]\mathsf{LTT}$)**.** *Closed terms typable in* $[\![2]\!]\mathsf{LTT}$ *are strongly normalizing with respect to* $(\rightsquigarrow)$*.*

Proof. We could reprove normalization from scratch, just how we did with $[\![\mathsf{S}]\!]\mathsf{TLC}$. However, here we can provide a much simpler proof which delegates normalization to another type system. To be more precise, this system $\mathcal{S}$ is Martin-Löf type theory with $\omega + \omega$ universes and first-class universe polymorphism allowed to range over first ω universes; we know it to be strongly normalizing [CITE]. Then, we define a translation function $[_]$ which transforms types and terms of $[\![2]\!]\mathsf{LTT}$ into types and terms of $\mathcal{S}$: it preserves all vanilla grammar formers and only translates lock $[_]_\text{🔒}$ into a function from the type of models $\mathcal{M}^{\ell'_0,\ldots,\ell'_\kappa}_{\ell_0,\ldots,\ell_\kappa}$ into interpreted type. Since κ is fixed, first-class universe polymorphism is enough to translate it into a function which accepts models for *any* list of levels of fitting length. Then, unlocking $[_]_\text{🔓}$ translates into application of this function to identity model of fitting level; interpretation translates into application.

Note that $t \rightsquigarrow t'$ implies $[t] \rightsquigarrow^+ [t']$; this concludes the proof since any infinite chain of reductions in our system would imply an existence of infinite chain of reductions in $\mathcal{S}$. □

Theorem 11 (Conservativity over MLTT)**.** *For MLTT context* Γ*, MLTT type* T *and* $[\![2]\!]\mathsf{LTT}$ *term* t*, if* $\Gamma \vdash t \Leftarrow T$*, then* $\Gamma \vdash_{\mathrm{MLTT}} \mathrm{nf}(t) \Leftarrow T$*.*

Proof. Thanks to type preservation, normal form $\mathrm{nf}(t)$ is typable in $[\![2]\!]\mathsf{LTT}$. Since Γ does not contain neither $\Box$-es nor 🔒-s, any subterm of type $\Box_\kappa T$ has form $[_]_\text{🔒}$. Since T is an MLTT type, every such locking is wrapped either in a $[_]_\text{🔓}$ or in a $[\![_]\!]$. Second case is a redex and is impossible in a normal form; $[_]_\text{🔓}$ is possible only in a locked context, so that means that there's another $[_]_\text{🔒}$ above it. Therefore $\mathrm{nf}(t)$ has no subterms of type $\Box_\kappa T$; then it has no unlocking and interpretation subterms and is an MLTT term and is typable in bidirectional MLTT (it is obtained from $[\![2]\!]\mathsf{LTT}$ by getting rid of rules involving modalities, locking and reinterpretation). □

4.3 Useful reinterpretations inside $[\![2]\!]\mathsf{LTT}$

Here we list some of the language features which reinterpretation enables without any additional language support.

4.3.1 Crude, but effective stratification

Due to McBride [18, 19], a technique of crude, but effective stratification works as follows: instead of providing complete support for level polymorphism, let's consider a *displacement operator* which, given a term t using universes $U_i, U_j, U_k, \ldots$, produces a term which is equal to t except for universes which are replaced with $U_{i+1}, U_{j+1}, U_{k+1}, \ldots$. The displaced term is also well-typed, however it is defined «one level higher». While not directly definable as a single operator inside $[\![2]\!]\mathsf{LTT}$, for each κ, ℓ, one can define displacement operators $\uparrow_\ell^\kappa \colon \mathcal{M}_{\kappa+0,\kappa+1,\ldots,\kappa+\ell}^{\kappa+1,\kappa+2,\ldots,\kappa+\ell+1}$ that shifts a boxed definition which uses universes up to $\mathcal{U}_\ell$ by κ levels up. In this model, category of contexts basic presheaves are shifted by κ in comparison to an identity model; all other term interpretations are identical to the ones given in an identity model.

A bit more details on definition of $\uparrow_\ell^\kappa$ are given in Appendix B.

4.3.2 Internal external parametricity

Demonstrated by Reynolds [21], parametricity is a property of a type system that, in simple words, «polymorphic function cannot meaningfully depend on generic inputs». Thus any polymorphic function of type σ satisfies a certain «free theorem» $[\![\sigma]\!]$ defined via relational interpretation of a type.

In recent works [4, 1], a streamlined definition is given for parametricity interpretation of dependently-typed systems dubbed «external parametricity». Once again, this relational interpretation is definable in our system *internally* with the help of a relational model $\mathrm{Para}_\ell : \mathcal{M}_{1,\ldots,\ell+1}^{2,\ldots,\ell+2}$. The shift by one level happens because type $T : \mathcal{U}_\ell$ in such a model is interpreted by a triple $(T_1\, T_2 : \mathcal{U}_\ell) \times (T_1 \to T_2 \to \mathcal{U}_\ell)$. Since interpreted functions do not depend on local context, we sidestep issues with relational context highlighted by Altenkirch and others [1], thus result of the interpretation is a proper proof of a free theorem.

A bit more details on definition of Para_ℓ are given in Appendix C.

4.3.3 Synthetic mathematics

As thoroughly discussed in a recent paper by Nawrocki and others [20], thanks to MLTT being an internal language of certain $(\infty, 1)$-categories, we can do *synthetic mathematics* inside it: prove a theorem inside MLTT about types; interpret it in a model of MLTT and yield a proof about objects in that model. It is easy to see that reinterpretation operator we have in $[\![2]\!]\mathsf{LTT}$ is sufficient to do synthetic mathematics in it and immediately apply results interpreted in a suitable model.

5 Discussion & related work

5.1 What was done before us

Original work by Lambek [17] sets the tone of the future research, establishing correspondence between type systems based on lambda calculus with various kinds of categories, starting with correspondence between simply typed lambda calculus and cartesian closed categories.

Conal Elliott's "Compiling to Categories" [10] shows how to interpret (some subset of) Haskell in arbitrary `Category` (again, definable in Haskell) via GHC compiler plugin. That paper is a direct inspiration for this one. There is an important difference, though: reinterpretation is internal to our type systems, unlocking new patterns in the usage of reinterpretation, including (but not limited to) universe polymorphism, external parametricity and more. However, Conal's approach is more flexible since his translation to categories requires desired category to only implement primitives actually used for translation of a particular term at hand.

Conor McBride's "Crude but effective stratification" [18, 19] (systemic treatment of this idea is given by Favonia and others [13]) introduces an eye-opening idea of using *displacement* as a lightweight form of universe polymorphism: to reuse term typed at a lower universe level, lift all levels in a term by adding a constant shift s. From the point of view advocated by this paper, you can see it as an act of reinterpreting an expression in an internal model of type system where universe at level i is interpreted as a universe at level $i + s$ (Section 4.3.1).

In "Parametricity and Dependent Types" by Jean-Philippe Bernardy, Patrik Jansson and Ross Paterson [4], authors introduce a direct translation procedure to yield parametricity theorems and proofs from polymorphic definitions in a dependently typed system. The definition of «external parametricity» we follow, though, is given in "Internal Parametricity, without an Interval" by Thorsten Altenkirch, Yorgo Chamoun, Ambrus Kaposi and Michael Shulman [1]. Once again, we can reformulate the translation they give as reinterpretation in an internal model. Altenkirch and others noted an issue with this approach: if we apply the external translation to the judgment with nonempty context, context becomes parametric, too; that's the reason they add `rel` operation and yield type system with *computational properties of higher observational type theory*. We sidestep the issue with context since reinterpretation is applied only to terms whose context is also «reinterpretable» and is not affected by the translation (Section 4.3.2).

5.2 What we could've done differently

5.2.1 Just use metaprogramming instead

In a recent paper "A Certifying Proof Assistant for Synthetic Mathematics in Lean", Wojciech Nawrocki and others [20] adapt *Compiling to categories* approach [10] to Lean's system with dependent types, using Categories-with-Families to represent internal models and employing Lean's powerful metaprogramming facilities instead of a compiler plugin. In doing so, they are able to provide a framework for doing synthetic mathematics in Lean, extracting proofs done in MLTT applied to more specific categories. Thus our results in Section 4.3.3 are close to the results shown in their paper. To add to that, their work is already implemented in practical proof assistant. Yet, our type system involves no metaprogramming since reinterpretation is internal to it, and it handles conversion rule. In addition, the focus is a bit different, where we use reinterpretation to express other polymorphic features of a language.

5.2.2 Computational modality of global elements

Since we introduced a family of modalities of reinterpretable terms $\Box_\ell$ in Section 4, one could hope to allow computation in boxed terms. Generalizing a bit, the following definition will be useful further:

Definition 11. A dependently-typed system with a type of functions ($\to$) has a *computational modality of global elements* $\Box$ if there is a one-hole term nec _ and an atomic two-hole term K _ _ such that the following rules are admissible:

$$\frac{\cdot \vdash T\,\mathrm{type}}{\Gamma \vdash \Box T\,\mathrm{type}} \qquad \frac{\cdot \vdash t : T}{\Gamma \vdash \mathrm{nec}\, t : \Box T} \qquad \frac{\Gamma \vdash f : \Box\,(S \to T) \qquad \Gamma \vdash t : \Box\, S}{\Gamma \vdash K\, f\, t : \Box T}$$

And, in addition, the following reduction is admissible: $K\,(\mathrm{nec}\,\lambda x.t)\,(\mathrm{nec}\,s) \rightsquigarrow^* \mathrm{nec}(t[s/x])$.

With an exception of a computational rule, these are exactly the properties of modalities $\Box_\ell$ from the original Section 4. The intuition behind the reduction is that, if we apply a "global function" to a "global argument", we then should be able to "inline" this application, what this reduction precisely does.

Then, a corresponding weakening of computational reinterpretation can be defined as follows.

Definition 12 (Modal computational reinterpretation)**.** We call a type operator $F[_]$ a modal computational reinterpretation of types if it is defined on boxed types (if $\Gamma \vdash \Box T\,\mathrm{Type}$, then $\Gamma \vdash F[T]\,\mathrm{Type}$), respects computation on types (if $T \to T'$, then $F[T] \to^* F[T']$) and computes as a primitive recursive function on normal forms of types. Analogously, term with a hole $[\![_]\!]$ is a modal computational reinterpretation of terms if it respects computation on terms, computes as a primitive recursive function on normal forms of terms and, if $\Gamma \vdash t : \Box T$, then $\Gamma \vdash [\![t]\!] : F[T]$.

Maybe, this definition of reintrerpretation is "nice"? Regrettably, there's a no-go theorem which is similar to naïve version 1 which holds in this case:

Theorem 12 (No-go theorem of modal reinterpretation)**.** *In a confluent type system with modal computational reinterpretation, the following should hold for arbitrary well-typed s and t:*

$$\mathtt{app}[\![\mathrm{nec}\,\lambda x.t]\!][\![\mathrm{nec}\,s]\!] \equiv [\![\mathrm{nec}(t[s/x])]\!]$$

where $\equiv$ is definitional equality and app *is how $[\![_]\!]$ acts on K.*

Proof. By direct example. We are given well-formed types $\cdot \vdash S, T$ type and well-typed terms $\cdot \vdash s : S$ and $\cdot, x : S \vdash t : T$. Consider the following term:

$$\Gamma, x : \Box(S \to T), y : \Box\, S \vdash [\![K\, x\, y]\!] : F[T]$$

Since K is atomic, term under interpretation is in normal form, hence the whole term computes to $\mathtt{app}[\![x]\!][\![y]\!]$. Substituting $\mathrm{nec}\,\lambda x.t$ for x and $\mathrm{nec}\,s$ for y, we get $\mathtt{app}[\![\mathrm{nec}\,\lambda x.t]\!][\![\mathrm{nec}\,s]\!]$.

On the other hand, if we perform substitution first, we would get $\Gamma \vdash [\![K\,(\mathrm{nec}\,\lambda x.t)(\mathrm{nec}\,s)]\!] : F[T]$. Since modal reinterpretation respects reduction, this reduces to $[\![\mathrm{nec}(t[s/x])]\!]$.

In a confluent type system with functions, multi-step reduction commutes with substitution, therefore two terms are definitionally equivalent. □

This, of course, doesn't prohibit us from using such a modality: as is with naïve interpretation, this approach works if we manage to constrain models to those where equalities hold definitionally.

Another benefit of modal computational reinterpretation is the following property:

Theorem 13 (Collapse of computational modalities)**.** *The following implication about $[\![2]\!]\mathsf{LTT}$ is true:*

$$\frac{\exists \kappa : \Gamma.\, {}_{\ell}\!\blacksquare \vdash_{\kappa} t : T}{\forall j \geqslant \mathrm{level}(T) : \Gamma.\, {}_{\ell}\!\blacksquare \vdash_{j} \mathrm{nf}(t) : \mathrm{nf}(T)}$$

We do not give a direct proof here, but the general idea is that any level higher than level of T can only appear in some redex of t or T. Then, if modalities were computational, it would be enough to use a single modality $\Box$ instead of using a family of modalities $\Box_\ell$: level of type under box would tell how many universe models do we need to provide.

But, again, it all comes down to defining the type of *definitional* internal models.

5.2.3 Interpreting interpretation

In Section 4, we introduced our "reinterpretable terms" modality in a very constraining way specifically so that we wouldn't have to provide interpretation of this modality inside internal models. Once again, the problem is with terms that try to interpret variables from CwF context; simplest example is $[\![\lambda x.[\![x]\!]_1]\!]_2$. In order to be able to interpret such term, we would have to define *what reinterpretation operator means inside an internal model*. This quickly gets out of hand: rather than just turning type of models into a (recursive? inductive?) type, type of models would become such a type that a type of one of its fields depends on the value of the whole model. Actually, this happened before: Kolomatskaia et al. [16] have already defined a type like this inside their *displayed type theory*: type SST of semi-simplicial types is defined coinductively in terms of values of itself[5]. While defining type of models in similar vein could be an interesting development of type system presented in this paper, we deem it out of scope for now.

6 Conclusions and further work

In retrospect, we managed to define two self-interpreting type systems, $[\![\mathsf{S}]\!]\mathsf{TLC}$ and $[\![2]\!]\mathsf{LTT}$, meaning that they have a notion of their own "internal models" which they then can use to reinterpret their own terms inside such models. In doing so, such type systems get new features which previously were defined as separate builtin quirks of a system. Basic metatheoretical properties are proven; for future work we have the following plans:

[5] More generally, Shulman dubs such types *modal codatatypes* [22].

- We might try making locking discipline in Section 3 more flexible, allowing more than one lock in a context. Most likely, this would require us to incorporate modalities in the definition of an "internal model" type and bring us closer to the truly reflective system.
- While our technique to reinterpret conversion rule in Section 4 works, a current approach to equality witness extraction is a *hack*. A proper solution would require to specify withess extraction as a redex and, possibly, incorporate certain *algebras of reduction strategies* to make this process more controllable in proof-relevant settings.
- A Categories-with-Families approach to specifying internal models here is, in a sense, a "deep" interface: while all of its facilities are properly used in interpretation, we have a low-hanging fruit of being able to interpret terms in arbitrary CwF context. This hints at a direct extension of our modality of reinterpretable terms into full-blown contextual types.
- Having implemented contextual types, it then becomes possible to handle "partial" reinterpretation of terms: if a term under interpretation doesn't use a certain feature of a type system, then you shouldn't have to specify it in a model! This would bring our language closer to Conal Elliott's approach of "Compiling to Categories" [10] and closer to *actual* type system for synthetic mathematics.
- It would be nice to make equality in $[\![2]\!]\mathsf{LTT}$ more extensional, most probably by making type system closer to Homotopy type theory [23].
- At some moment we would have to pay due diligence: provide mechanized proofs of metatheoretical properties of systems and an implementation of $[\![2]\!]\mathsf{LTT}$ in a practical proof assistant.

References

[1] Thorsten Altenkirch et al. "Internal Parametricity, without an Interval". In: *Proc. ACM Program. Lang.* 8.POPL (Jan. 2024). DOI: 10.1145/3632920. URL: https://doi.org/10.1145/3632920.

[2] Nada Amin and Tiark Rompf. "Collapsing towers of interpreters". In: *Proc. ACM Program. Lang.* 2.POPL (Dec. 2017). DOI: 10.1145/3158140. URL: https://doi.org/10.1145/3158140.

[3] Andrej Bauer. "On Self-Interpreters for System T and Other Typed λ-Calculi". Unpublished manuscript. Jan. 2016. URL: https://math.andrej.com/wp-content/uploads/2016/01/self-interpreter-for-T.pdf.

[4] Jean-Philippe Bernardy, Patrik Jansson, and Ross Paterson. "Parametricity and dependent types". In: *Proceedings of the 15th ACM SIGPLAN International Conference on Functional Programming*. ICFP '10. Baltimore, Maryland, USA: Association for Computing Machinery, 2010, pp. 345–356. ISBN: 9781605587943. DOI: 10.1145/1863543.1863592. URL: https://doi.org/10.1145/1863543.1863592.

[5] Marc Bezem et al. *A Note on Generalized Algebraic Theories and Categories with Families*. 2021. arXiv: 2012.08370 [math.CT]. URL: https://arxiv.org/abs/2012.08370.

[6] Lars Birkedal et al. "Modal dependent type theory and dependent right adjoints". In: *Mathematical Structures in Computer Science* 30.2 (2020), pp. 118–138. DOI: 10.1017/S0960129519000197.

[7] Matt Brown and Jens Palsberg. "Breaking through the normalization barrier: a self-interpreter for f-omega". In: *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. POPL '16. New York, NY, USA: Association for Computing Machinery, 2016, pp. 298–311. DOI: 10.1145/2837614.2837623. URL: https://dl.acm.org/doi/10.1145/2837614.2837623.

[8] Simon Castellan, Pierre Clairambault, and Peter Dybjer. *Categories with Families: Unityped, Simply Typed, and Dependently Typed*. 2020. arXiv: 1904.00827 [cs.LO]. URL: https://arxiv.org/abs/1904.00827.

[9] Dominique Clément et al. "A simple applicative language: mini-ML". In: *Proceedings of the 1986 ACM Conference on LISP and Functional Programming*. LFP '86. Cambridge, Massachusetts, USA: Association for Computing Machinery, 1986, pp. 13–27. ISBN: 0897912004. DOI: 10.1145/319838.319847. URL: https://doi.org/10.1145/319838.319847.

[10] Conal Elliott. "Compiling to categories". In: *Proc. ACM Program. Lang.* 1.ICFP (Aug. 2017). DOI: 10.1145/3110271. URL: https://doi.org/10.1145/3110271.

[11] Thiago Felicissimo. "Generic Bidirectional Typing for Dependent Type Theories". In: *ACM Trans. Program. Lang. Syst.* 47.1 (Apr. 2025). ISSN: 0164-0925. DOI: 10.1145/3715095. URL: https://doi.org/10.1145/3715095.

[12] Daniel Gratzer, Jonathan Sterling, and Lars Birkedal. "Implementing a modal dependent type theory". In: *Proc. ACM Program. Lang.* 3.ICFP (July 2019). DOI: 10.1145/3341711. URL: https://doi.org/10.1145/3341711.

[13] Kuen-Bang Hou (Favonia), Carlo Angiuli, and Reed Mullanix. "An Order-Theoretic Analysis of Universe Polymorphism". In: *Proc. ACM Program. Lang.* 7.POPL (Jan. 2023). DOI: 10.1145/3571250. URL: https://doi.org/10.1145/3571250.

[14] W. Howard. "The Formulae-as-Types Notion of Construction". In: *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*. Ed. by J. Seldin and J. Hindley. Academic Press, 1980, pp. 479–490.

[15] Akira Kawata and Atsushi Igarashi. *A Dependently Typed Multi-Stage Calculus*. 2021. arXiv: 1908.02035 [cs.PL]. URL: https://arxiv.org/abs/1908.02035.

[16] Astra Kolomatskaia and Michael Shulman. "Displayed type theory and semi-simplicial types". In: *Mathematical Structures in Computer Science* 35 (2025), e34. DOI: 10.1017/S096012952510025X.

[17] Joachim Lambek. "From λ-calculus to cartesian closed categories". In: *To H. B. Curry: Essays on Combinatory Logic, Lambda-Calculus and Formalism*. Ed. by J. P. Seldin and J. R. Hindley. London: Academic Press, 1980, pp. 375–402.

[18] Conor McBride. *Crude but Effective Stratification*. 2011. URL: https://mazzo.li/epilogue/index.html%5C%3Fp=857%5C&cpage=1.html (visited on 07/09/2026).

[19] Conor McBride. *Crude but Effective Stratification*. 2012. URL: https://personal.cis.strath.ac.uk/conor.mcbride/Crude.pdf (visited on 07/09/2026).

[20] Wojciech Nawrocki et al. "A Certifying Proof Assistant for Synthetic Mathematics in Lean". In: *Proceedings of the 15th ACM SIGPLAN International Conference on Certified Programs and Proofs*. CPP '26. Rennes, France: Association for Computing Machinery, 2026, pp. 88–103. ISBN: 9798400723414. DOI: 10.1145/3779031.3779087. URL: https://doi.org/10.1145/3779031.3779087.

[21] John C Reynolds. "Types, abstraction and parametric polymorphism". In: *Information Processing 83*. Ed. by R. E. A. Mason. North-Holland. 1983, pp. 513–523.

[22] Michael Shulman. *Modal datatypes and codatatypes*. 2026. URL: https://gwaithimirdain.github.io/2026/06/02/modal-data-and-codata.html (visited on 07/09/2026).

[23] The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study: https://homotopytypetheory.org/book, 2013.

A A type of internal models of $[\![2]\!]\mathsf{LTT}$

TODO: finish

A comprehension $[x : A, y : B, z : C]$ denotes a right-nested Σ-type. $\{x : A\} \to B$ is a syntactic sugar meaning that argument x is always inferred from B rather than provided explicitly. $\{x\} \to B$ means that type of x is inferred from B. Transports along previously introduced equalities are omitted. Since our language doesn't yet have η-equivalence, uniqueness properties are skipped.

$$\mathcal{M}^{\ell'_0,\ldots,\ell'_\kappa}_{\ell_0,\ldots,\ell_\kappa} = [$$

A.1 Base CwF part

$$
\begin{aligned}
&\mathcal{C} : \mathcal{U}_{\max(\ell'_0,\ldots,\ell'_\kappa)},\\
&\triangleright : \mathcal{C} \to \mathcal{C} \to \mathcal{U}_{\max(\ell_0,\ldots,\ell_\kappa)},\\
&\mathrm{id} : \{\Gamma : \mathcal{C}\} \to \Gamma \triangleright \Gamma,\\
&\circ : \{\Gamma\,\Delta\,\Xi\} \to (\Delta \triangleright \Xi) \to (\Gamma \triangleright \Delta) \to (\Gamma \triangleright \Xi),\\
&\mathrm{id}{-}\circ : \{\Gamma, \Delta\} \to \{\sigma : \Gamma \to \Delta\} \to \mathrm{id} \circ \sigma = \sigma,\\
&\circ{-}\mathrm{id} : \{\Gamma, \Delta\} \to \{\sigma : \Gamma \to \Delta\} \to \sigma \circ \mathrm{id} = \sigma,\\
&\circ{-}\mathrm{assoc} : \{\rho\,\sigma\,\tau\} \to \rho \circ (\sigma \circ \tau) = (\rho \circ \sigma) \circ \tau,\\
&\cdot : \mathcal{C},\\
&! : \{\Gamma\} \to \Gamma \triangleright \cdot,
\end{aligned}
$$

A.2 Common part for each level j

$$
\begin{aligned}
&\mathrm{Ty}_j : \mathcal{C} \to \mathcal{U}_{\ell'_j},\\
&_[_] : \{\Gamma\,\Delta\} \to \mathrm{Ty}_j\Gamma \to \Delta \triangleright \Gamma \to \mathrm{Ty}_j\Delta,\\
&\mathrm{sub/id} : \{T\} \to T[\mathrm{id}] = T,\\
&\mathrm{sub/comp} : \{T\,\sigma\,\tau\} \to T[\sigma \circ \tau] = T[\sigma][\tau],\\
&\mathrm{Tm}_j : (\Gamma : \mathcal{C}) \to \mathrm{Ty}_j\,\Gamma \to \mathcal{U}_{\ell_j},\\
&_[_] : \{\Gamma\,\Delta\,T\} \to \mathrm{Tm}_j\,\Gamma\,T \to (\sigma : \Delta \triangleright \Gamma) \to \mathrm{Tm}_j\,\Delta\,T[\sigma],\\
&\mathrm{sub/id} : \{T\} \to \{t : \mathrm{Tm}_\,T\} \to t[\mathrm{id}] = t,\\
&\mathrm{sub/comp} : \{T\,\sigma\,\tau\} \to \{t : \mathrm{Tm}_j\,_\,T\} \to t[\sigma \circ \tau] = t[\sigma][\tau],\\
&(.) : (\Gamma : \mathcal{C}) \to \mathrm{Ty}_j\,\Gamma \to \mathcal{C},\\
&p : \{\Gamma\,T\} \to \Gamma.T \triangleright \Gamma,\\
&q : \mathrm{Tm}\,\Gamma.T\,T[p],\\
&\langle,\rangle : \{\Delta\,\Gamma\} \to (\sigma : \Delta \triangleright \Gamma) \to \mathrm{Tm}\,\Delta\,T[\sigma] \to \Delta \triangleright \Gamma.T,\\
&\mathrm{ext/l} : \{\sigma\,t\} \to p \circ \langle\sigma, t\rangle = \sigma,\\
&\mathrm{ext/r} : \{\sigma\,t\} \to q[\langle\gamma, t\rangle] = t,\\
&\mathrm{Nat}_j : \{\Gamma\} \to \mathrm{Ty}_j\,\Gamma,\\
&0 : \{\Gamma\} \to \mathrm{Tm}_j\,\Gamma\,\mathrm{Nat}_j,\\
&S : \{\Gamma\} \to \mathrm{Tm}_j\,\Gamma\,\mathrm{Nat}_j \to \mathrm{Tm}_j\,\Gamma\,\mathrm{Nat}_j,\\
&\mathrm{Id} : \{\Gamma\} \to (T : \mathrm{Ty}_j\,\Gamma) \to \mathrm{Tm}_j\,\Gamma\,T \to \mathrm{Tm}_j\,\Gamma\,T \to \mathrm{Ty},\\
&\mathrm{refl} : \{\Gamma\,T\} \to \{t\,t' : \mathrm{Tm}_j\,\Gamma\,T\} \to t = t' \to \mathrm{Tm}_j\,\Gamma\,(\mathrm{Id}\,T\,t\,t'),
\end{aligned}
$$

A.3 Common part for each j less then k

Here, definition of a model contains internalization of universes and of type family El at internal level $j + 1$.

A.4 Common part for each i, j

Here, definition of a model contains eliminators for model of naturals and for model of an identity type; additionally, models of Π- and Σ- types for internal level $\max(i, j)$ are defined here.

A.5 End of definition of a model type

]

B Parts of a definition of a displacement model

TODO: finish

$$
\begin{aligned}
\uparrow_\ell^\kappa = \ & [\\
& \mathcal{C} = \mathcal{U}_{\ell+\kappa}, \triangleright = (\to), \mathrm{id} = \lambda x.x, \circ = \lambda f g x.f(gx), \mathrm{id}{-}\circ = \mathrm{refl}, \circ{-}\mathrm{id} = \mathrm{refl}, \\
& \ldots, \\
& \mathrm{Ty}_j = \lambda \Gamma.\Gamma \to \mathcal{U}_{j+\kappa}, \mathrm{Tm}_j = \lambda \Gamma\, f.(\gamma : \Gamma) \to\ f\,\gamma, \\
& \ldots \\
&]
\end{aligned}
$$

C Parts of a definition of a parametricity model

TODO: finish

$$
\begin{aligned}
\mathrm{Para}_\ell \ = \ & [\\
& \mathcal{C} = (\Gamma\,\Gamma' : \mathcal{U}_{\ell+1}) \times (\Gamma \to \Gamma' \to \mathcal{U}_{\ell+1}), \\
& \triangleright = \lambda(\Gamma, \Gamma', Q)(\Delta, \Delta', R).(f : \Gamma \to \Delta) \times (f' : \Gamma' \to \Delta') \times ((\gamma\,\gamma') \to Q\,\gamma\,\gamma' \to R\,(f\,\gamma)\,(f'\,\gamma')), \\
& \mathrm{id} = (\lambda x.x, \lambda x.x, \lambda x.x), \\
& \circ = \lambda(f, f', q)(g, g', r)(x, x', s).(f(g(x)), f'(g'(x')), q(r(s))), \\
& \mathrm{id}{-}\circ = \mathrm{refl}, \circ{-}\mathrm{id} = \mathrm{refl}, \\
& \ldots, \\
&]
\end{aligned}
$$